

ML on ZX Spectrum: Functional Programming on Z80

Why This Matters

Zero-Cost Abstractions

The Minigame Engine (226 bytes)

Collision, Scoring, Health — All Pure Functions

The Parser Combinator (498 bytes)

427 Assertions, Zero Runtime Tests

The Compiler Pipeline

What's Next

Try It

ML on ZX Spectrum: Functional Programming on Z80

Algebraic data types, pattern matching, pipe operators — on a ZX Spectrum. 3.5 MHz. 48 KB RAM. 1982.

Frill is an ML-style functional language that compiles to native Z80 machine code. Not interpreted. Not bytecoded. Directly compiled to the same instruction set that runs Pac-Man, ZX Spectrum games, and CP/M business software.

Here's a complete traffic light controller:

```
type Light = Red | Yellow | Green
```

```
let next_light (s : u8) : u8 =  
  match s with  
  | Red    -> 2    (* → Green *)  
  | Yellow -> 0    (* → Red  *)  
  | Green  -> 1    (* → Yellow *)  
end
```

This compiles to **175 bytes** of Z80 machine code. The entire state machine, with exhaustive match verification at compile time — the compiler **proves** every state is handled.

Why This Matters

The Z80 was designed in 1976. It has: - 3.5 MHz clock (your coffee machine is faster) - 64 KB address space (smaller than a favicon) - 8-bit registers (counts to 255, then wraps) - No stack frames, no heap, no OS

Most Z80 programming is done in assembly or C. Functional programming? That's for modern hardware with gigabytes of RAM and garbage collectors. Right?

Wrong.

Zero-Cost Abstractions

Every Frill abstraction compiles away completely:

ADT variants → integer tags (Red=0, Yellow=1, Green=2). No vtables. No heap allocation. One byte.

Pattern matching → conditional jumps. The compiler generates the same CP / JR Z sequence a human would write in assembly.

Pipe operators → function calls. `x |> double |> inc` compiles to `CALL double / CALL inc`. The pipe is syntactic sugar that costs zero runtime overhead.

Let bindings → register allocation. `let a = x + 1 in ...` becomes `ADD A, 1`. The variable `a` lives in a CPU register, not memory.

```
let pipeline_demo (x : u8) : u8 = x |> double |> inc |> square
```

This compiles via Z3 SMT solver to **provably optimal** register assignments. The compiler doesn't guess — it mathematically proves the best register allocation.

The Minigame Engine (226 bytes)

A complete game logic library, purely functional:

```
type Entity = Player | Enemy | Bullet | Coin | Wall
```

```
let entity_char (e : u8) : u8 =  
  match e with  
  | Player -> 64    (* '@' *)  
  | Enemy  -> 35    (* '#' *)
```

```

| Bullet -> 42    (* '*' *)
| Coin   -> 36    (* '$' *)
| Wall   -> 219   (* block char *)
end

let is_solid (e : u8) : u8 =
  match e with
  | Player -> 0
  | Enemy  -> 1
  | Bullet -> 0
  | Coin   -> 0
  | Wall   -> 1
  end

```

The compiler enforces **exhaustive matching** — miss a variant and it refuses to compile:

```
Error: non-exhaustive match on Entity: missing Wall
```

This catches bugs at compile time that would be runtime crashes in C. On a machine with no debugger, no stack trace, no exception handler — compile-time safety is everything.

Collision, Scoring, Health — All Pure Functions

```

let manhattan (ax : u8) (ay : u8) (bx : u8) (by : u8) : u8 =
  abs_diff ax bx + abs_diff ay by

let take_dmg (hp : u8) (dmg : u8) : u8 =
  if hp < dmg then 0 else hp - dmg

let apply_score (base : u8) (combo : u8) : u8 =
  base * combo_mult combo

```

Each function is pure — no mutation, no side effects. The entire game state is computed fresh each frame. On Z80, this compiles to tight register-only code. The `combo_mult` call inlines to a comparison chain. Total binary: **226 bytes**.

The Parser Combinator (498 bytes)

Functional parsing on Z80. Character classification, tokenization, expression evaluation:

```

let parse_digit (c : u8) : u8 =
  if is_digit c == 1 then c - 48 else 255

let parse_hex_byte (hi : u8) (lo : u8) : u8 =

```

```

let h = parse_hex hi in
let l = parse_hex lo in
if h == 255 then 255
else if l == 255 then 255
else h * 16 + l

let eval_op (a : u8) (op : u8) (b : u8) : u8 =
  if op == 43 then a + b      (* + *)
  else if op == 45 then a - b (* - *)
  else if op == 42 then a * b (* * *)
  else if op == 47 then
    if b == 0 then 0 else a / b
  else 0

```

parse_hex_byte 70 70 → 255 (0xFF). Verified by 45 compile-time assertions. The parser fits in **498 bytes** — smaller than this paragraph.

427 Assertions, Zero Runtime Tests

Frill assertions run at compile time:

```

assert factorial 5 == 120
assert gcd 12 8 == 4
assert pipeline_demo 3 == 49

```

The compiler executes these during compilation on the MIR2 virtual machine. If any assertion fails, compilation stops. **427 assertions** across 16 examples — all verified before a single byte of Z80 code is emitted.

The Compiler Pipeline

Frill source → Parser → HIR → MIR2 → VIR (Z3 solver) → Z80 ASM → Binary

↓
PBQP fallback
(if Z3 times out)

The VIR backend uses a Z3 SMT solver for joint instruction selection + register allocation. It literally asks a theorem prover: “what is the mathematically optimal way to assign registers for this function?” For functions that are too complex for Z3 (>100 variables), it falls back to PBQP heuristic allocation — still correct, just not provably optimal.

The result: **-71% code size vs SDCC** (the standard Z80 C compiler) across a benchmark corpus.

What's Next

Frill already has 38 features including: - Algebraic data types with payload - Exhaustive pattern matching with guards - Pipe operator `|>` and composition `>>` - Lambda expressions (zero-cost) - Type classes - QTT linearity markers (! = must use, ~ = affine) - Property testing at compile time - While/for loops with mutation (peek/poke)

Coming soon: - **COBOL frontend** using Frill's ADT for COBOL PICTURE types - **BCD packed decimal** native type (Z80 DAA instruction) - **GPU-precomputed multiplication tables** via `#embed`

Try It

```
git clone https://github.com/oisee/minz
cd minz/minzc && go build -o mz ./cmd/minzc
./mz examples/frill/state_machine.frl -o out.a80
```

175 bytes. A traffic light controller. In ML. On ZX Spectrum.

MinZ v0.23.0 — 8 frontend languages, C23 conformant, Z3 optimal codegen, 501 GPU-precomputed arithmetic sequences. The world's most sophisticated compiler for the world's simplest CPU.